TradeIO

D1 Detailed Design

Trading Journal and Behavioral Analytics Platform

Senior Design

Assignment #4: Design Diagrams · Design Stage: D1

Team Members

Ishaa Jain

Samarth Prajapati

September 30, 2026

Project Goal

TradeIO is a trading journal designed to help traders record trading activity, reflect on execution and psychology, monitor risk and discipline, and receive useful performance and behavioral insights.

Contents

1 Header, Scope, and Conventions **2**

2 Data Model (D1 Diagram) **3**

3 Core Algorithms **5**

3.1 Algorithm A: Risk and Discipline Evaluation 5

3.2 Algorithm B: Behavioral and Psychology Summary 6

4 Build-versus-Reuse Decisions **7**

5 API Contract **8**

6 Technology Choices with Justification **10**

7 D0-to-D1 Traceability **12**

1. Header, Scope, and Conventions

Project Title

TradeIO: Trading Journal and Behavioral Analytics Platform

Goal Statement

TradeIO is a trade journal for traders that provides the ability to record trading actions, evaluate execution and psychology, track risks and discipline, and gain valuable insights about trading. This software is aimed at helping users not only analyze their trading outcomes but also to comprehend what behavior led to those outcomes.

Team Members

Ishaa Jain and Samarth Prajapati

D1 Scope

In D1, we chose to elaborate further on two components of D0: **C5 Data Storage** and **C4 Risk & Behavior Analysis**. The reason for choosing C5 was very simple, as every component of the TradeIO requires some interaction with its data model. C4 is crucial for the system, and failure to properly perform checks may result in the fact that the user misses a limit violation or gets used to ignoring alerts that were not supposed to occur in the first place. The remaining components of D0 will be addressed in D2 (C1 User Interface, C2 Application/API Layer, C3 Trade & Journal Processing and the internals of the external services).

Figure Conventions

- **Rectangles** correspond to persistent objects/tables. In the shaded part, you will find the name of the object while the white part contains the attributes.
- **PK** indicates a primary key, **FK** is for a foreign key and **UQ** indicates a uniqueness constraint. An **?** indicates the nullability of the attribute.
- **Solid lines** are used to indicate persistent relationships while the **dashed line** represents an optional relationship.
- **Cardinality** is indicated by each end of the line (**1**, **0..1**, **0..***).
- Attributes that affect system behavior only have been taken into consideration, while implementation-related attributes have not.
- External systems will be modeled as database entities if there is some state that TradeIO needs to keep locally.

2. Data Model (D1 Diagram)

We decided to choose the **relational data model**. Virtually all data in TradeIO belong to some other entity: users have accounts, accounts have trades, and journal entries have references to trades. Ownership relations can be easily translated into table structures and foreign keys. We will also require referential integrity, query scope limited to a single user, safe updates, and fast retrieval by time range. This is precisely what relational databases excel at and exactly what C5 Data Storage does in D0.

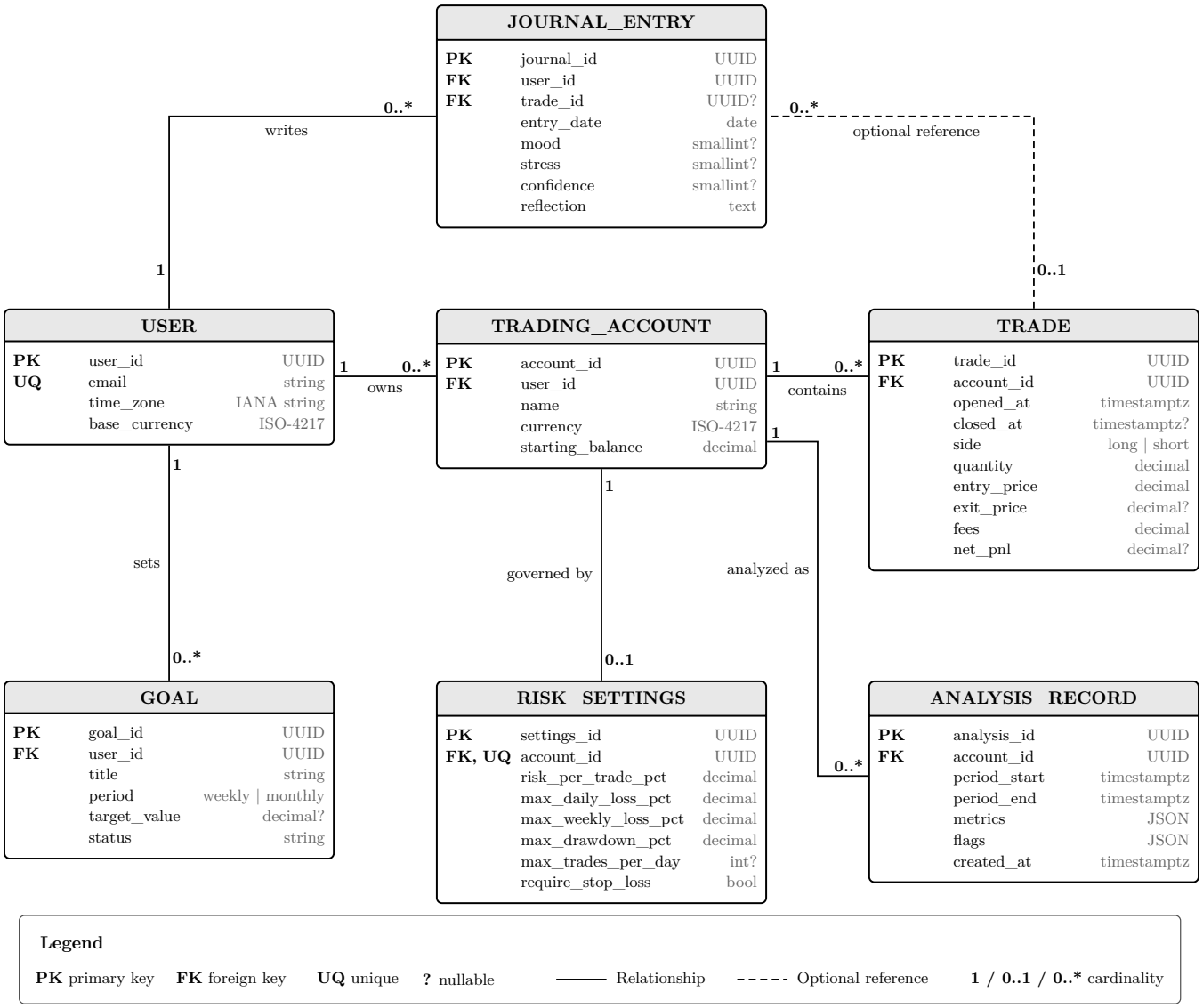


Figure 1: D1 Relational Data Model for C5 Data Storage and C4 Risk & Behavior Analysis

Relationship and Cardinality Decisions

Relationship	Cardinality	Reason
User – Trading Account	1 to 0..*	A user may maintain multiple trading accounts; each account has one owner.
Trading Account – Trade	1 to 0..*	Every trade belongs to one account, while an account can contain many trades.
User – Journal Entry	1 to 0..*	A user can create many private journal entries.
Trade – Journal Entry	0..1 to 0..* (optional)	A reflection may describe a specific trade or may be a general daily/weekly reflection.
Trading Account – Risk Settings	1 to 0..1	One active set of risk settings governs an account at a time.
Trading Account – Analysis Record	1 to 0..*	Analysis is stored by account and time period, allowing historical comparisons.
User – Goal	1 to 0..*	A user may define multiple weekly or monthly goals.

Structural Decisions

Decision	Justification
Trading Account is an entity rather than a User attribute	A user can trade from more than one account, and each account has its own currency, balance, risk settings, and history. Folding this into USER would break as soon as someone adds a second account.
Risk Settings are separate from Trading Account	Risk rules change on their own schedule, separately from the account's basic details, and C4 reads them directly every time it runs an evaluation.
Analysis Record is a separate entity	Keeping derived results apart from raw trades means the source data is never overwritten. If we change a formula later, we can simply recompute.
Journal Entry is separate from Trade	Plenty of reflections aren't about a single trade. A user might write about a whole day or week, or about a day they didn't trade at all, and that still needs to be saved.
Goal is an entity rather than a text field	Goals have their own period, target, status, and lifecycle. A single text field couldn't hold several goals or track progress on any of them.

Indexes

Index	Reason
`TRADE(account_id, opened_at DESC)`	Supports fast account history, dashboard ranges, trade-count checks, and time-based analytics.
`TRADE(account_id, closed_at) WHERE closed_at IS NOT NULL`	Supports realized P&L calculations without repeatedly scanning open trades.
`JOURNAL_ENTRY(user_id, entry_date DESC)`	Supports weekly psychology summaries and journal-history retrieval.
`ANALYSIS_RECORD(account_id, period_start, period_end)`	Supports fast retrieval of stored analysis for a specific account and time window.
`GOAL(user_id, period, status)`	Supports dashboard lookup of active weekly/monthly goals.

3. Core Algorithms

3.1 Algorithm A: Risk and Discipline Evaluation

What it is and the problem it solves. This is the check that decides whether a trader has broken one of their own risk or discipline rules. It pulls the account's trades, compares them against the limits the user configured, and raises flags. We treat it as correctness-critical. A bug here either hides a real breach or cries wolf, and both are bad for a tool people are supposed to trust.

Template Item	Specification
Inputs	`accountId: UUID`, `now: timestamptz`, `riskSettings: RiskSettings`, `userTimeZone: IANA string`.
Outputs	`RiskEvaluation { dailyTradeCount: int, realizedDailyPnl: decimal, dailyLossPct: decimal, maxDrawdownPct: decimal, flags: RiskFlag[] }`.
Computation	Figure out the start and end of the current day from the user's perspective in their own timezone and convert those boundaries to UTC. Then do a lookup on the trades within that period, calculate the net P&L, and find out how many trades there are. Finally, compare that to the threshold values and generate the risk or discipline flags.
Expected Complexity	Using the composite index on trades, the look-up complexity is approximately $O(\log N + K)$, with N being the total number of trades for the account and K the number of trades within the queried window. The average number of trades is about 2,500, which is very small. Even at 250,000 trades (a hundred times more), the performance penalty is negligible.
Why This Approach	We offloaded the task of range checking the dates to the database instead of bringing the entire history of an account to memory. The use of the index makes data migration easier and maintains a single point of truth. We also considered maintaining a count per day, but it is easily possible to get it out of sync with modifications and imports.
Edge Cases	In case there are no trades, all values will be equal to zero. Pending trades are not counted as part of the realized P&L until some subsequent criteria modify this behavior. Achieving a limit boundary exactly is considered as achieving the limit itself. An incomplete transaction history entry is not skipped if the calculation does not actually require this data. Invalid parameters generate an error instead of a guess. Day boundaries always use IANA timezones.

3.2 Algorithm B: Behavioral and Psychology Summary

What it is and the problem it solves. This calculation matches the result of trade with what the user writes down in their journal, thereby forming a behavioral description. This is a deterministic one. It may be presented to the user as such or provided to the optional AI Insight Service through the interface D0 I8 for more instructional text.

Template Item	Specification
Inputs	`accountId: UUID`, `periodStart: timestamptz`, `periodEnd: timestamptz`, `trades: TradeRecord[]`, `journal: JournalRecord[]`.
Outputs	`BehaviorSummary { tradeCount: int, winRatePct: decimal, avgMood: decimal|null, avgStress: decimal|null, avgConfidence: decimal|null, flags: string[] }`.
Computation	Filter out both records by the specified period, calculate basic performance indicators, and find an average value of psychological scores if possible. After that, try to spot the patterns that can be revealed using simple rules, such as losses occurring after high psychological pressure or repetitive violation of the same rule.
Expected Complexity	For T trades and J journal entries in the period, the complexity is $O(T + J)$. A week or month's worth of data is small. Even if the history of the account expands 100 times, the complexity is determined by the period, since only the data in that period will be queried.
Why This Approach	Maintaining the core summary deterministic will ensure that it continues working even when the AI service is either offline or slow because the AI simply provides additional explanation and coaching on top of the core summary. This is consistent with the D0 decision.
Edge Cases	No data means a blank summary, no guessing here. No psychology scores are included in average calculations. Duplicate records are rejected by unique record ID number. Zero is never the divisor in win percentage calculations. In case of ties between two behavior flags, both are selected.

4. Build-versus-Reuse Decisions

Piece	Decision	Library/Service	License	Reason
Risk and discipline rules	Build	TradeIO TypeScript logic	Project-owned	These rules are product-specific and directly implement TradeIO behavior.
Behavior summary logic	Build	TradeIO TypeScript logic	Project-owned	The deterministic psychology/performance combination is part of the project's core design.
Relational database	Reuse	PostgreSQL on Neon	PostgreSQL License; Neon service terms	Mature transactions, constraints, indexing, and managed hosting fit C5.
Database access	Reuse	Drizzle ORM	Apache-2.0	Provides type-safe schema/query access while preserving relational SQL behavior.
Authentication	Reuse	Clerk	Proprietary hosted service	Authentication is security-sensitive and should not be hand-rolled.
Date/time-zone handling	Reuse	JavaScript Intl / platform support	Platform standard	Avoids custom date parsing and supports IANA time zones.
Notification delivery	Reuse	External notification provider	Provider-specific	TradeIO should build alert-generation logic, not external mail infrastructure.
AI coaching	Reuse behind adapter	External AI Insight Service	Provider-specific	Optional coaching remains separate from deterministic analytics.
UI framework	Reuse + build	React / Next.js / Tailwind CSS	Permissive open-source licenses	Mature web stack lets the team focus effort on TradeIO workflows.

5. API Contract

This document details the areas of C4 and C5 exposed by the Application/API Layer. An authenticated session is required for each protected endpoint. The server determines the user based on the session, and all queries of accounts are restricted to only those accounts owned by the user. Monetary values are sent as strings to prevent rounding due to binary floating point.

Endpoint	Inputs	Success Output	Errors
`POST /api/v1/trades`	`accountId UUID` required; `openedAt ISO-8601` required; `side long|short`; `quantity >0`; `entryPrice >0`; optional `exitPrice`, `leverage`, `notes`.	`201 { tradeId: string, status: string }`	400 validation; 401 unauthenticated; 403 account not owned; 500 storage failure.
`GET /api/v1/accounts/{id}/analysis`	`id UUID`; optional `from`, `to` ISO-8601.	`200 { metrics: object, flags: string[], behaviorSummary: object }`	400 invalid range; 401; 403; 404 account; 500 analysis/storage failure.
`PUT /api/v1/accounts/{id}/risk-settings`	`maxDailyLossPct decimal [0,100]`; `maxWeeklyLossPct decimal [0,100]`; `maxDrawdownPct decimal [0,100]`; optional `maxTradesPerDay int >=1`; `requireStopLoss bool`.	`200 { saved: boolean, effectiveAt: string }`	400 validation; 401; 403; 404; 500.
`GET /api/v1/trades`	`accountId UUID` required; optional `from/to`; `limit int [1,100]` default 50; optional `cursor`.	`200 { items: TradeDTO[], nextCursor: string|null }`	400 range/cursor; 401; 403; 404; 500.
`POST /api/v1/journal`	`entryDate date`; optional `tradeId UUID`; optional `mood/stress/confidence smallint`; `reflection string`.	`201 { journalId: string, createdAt: string }`	400 validation; 401; 403 invalid trade ownership; 500.

Exact Response Shapes

```
TradeDTO = {
  id: string,                 // UUID
  accountId: string,          // UUID
  openedAt: string,           // ISO-8601
  closedAt: string | null,    // ISO-8601
  side: "long" | "short",
  quantity: string,           // decimal units
  entryPrice: string,         // decimal currency/unit
  exitPrice: string | null,
  fees: string,               // decimal currency
  netPnl: string | null       // decimal currency
}

AnalysisResponse = {
  metrics: {
    tradeCount: number,
    winRatePct: string,
```

```
    realizedPnl: string,
    maxDrawdownPct: string
  },
  flags: string[],
  behaviorSummary: {
    avgMood: string | null,
    avgStress: string | null,
    avgConfidence: string | null
  }
}
```

Example Request and Response

```
POST /api/v1/trades
Content-Type: application/json

{
  "accountId": "c2e5f634-39c2-41bd-9f56-53dc9f7e1790",
  "openedAt": "2026-09-30T14:30:00-04:00",
  "side": "long",
  "quantity": "0.05",
  "entryPrice": "2650.50",
  "exitPrice": "2661.00",
  "leverage": "10",
  "notes": "Followed planned entry and stop."
}

HTTP/1.1 201 Created

{
  "tradeId": "9f42d4cf-c1d2-42af-aac9-610c4bcc2826",
  "status": "stored"
}
```

Versioning

The version of the API is contained in the URL `/api/v1`. A change is considered to be breaking if it involves removing or renaming a documented property or endpoint, changing the type, units, or meaning of a field, or any documented error/status handling that existing clients cannot handle.

6. Technology Choices with Justification

Database: PostgreSQL on Neon

Team skill fit: We have both used SQL and relational databases before, and TradeIO's schema consists of tables, keys, transactions, and indexes, hence it was an obvious choice. **Licensing:** PostgreSQL is licensed under the liberal PostgreSQL license and Neon is the hosted service that we use under its license. **Community support:** PostgreSQL is very well-known, having been around for decades; its documentation and community are vast. **Performance:** Our most intensive queries include account lookups and range queries on time, which are very well-suited to indexing and transactions ensure consistent writes. **Cost and Hosting:** With Neon, we get fully-managed and serverless Postgres database without the need to run our own database server for the capstone project. We also considered using MongoDB but we have found that TradeIO's data model has many relations that need to be expressed.

Backend: Next.js Server/API Routes with TypeScript

Team skill fit: The use of TypeScript in the backend allows us to use one language for requests, validations, API logic, and the code of analysis, which is crucial when working with two people on the project. **Licensing:** Next.js is licensed under MIT and TypeScript is under Apache-2.0. **Community support:** They are both popular and have good documentation and support. **Performance:** API routes are sufficient for the traffic of a senior design project, and all analysis is done using indexed queries rather than large in-memory scanning. **Cost and Hosting:** The backend is hosted along with the frontend in Vercel, thus not requiring any additional infrastructure. We thought about a separate backend in Express/Spring, but that would require hosting an additional server without needing any scaling at all.

Front End: React/Next.js with Tailwind CSS

Team skill fit: This is the stack we build with, which makes iteration fast for trade entry, dashboards, journals, goals, and analysis views. **Licensing:** All three frameworks mentioned above come with permissive licenses. **Community support:** Large communities exist around all three, with lots of documentation. **Performance:** Interactive forms, tables, charts, and dashboards are handled seamlessly. **Cost and hosting:** No per-seat license, and it all deploys to Vercel in one piece. Accessibility is now considered a basic requirement instead of a nice-to-have: Semantic HTML, keyboard support, labeled form elements, visual feedback on focused elements, and status messages that do not use color alone.

Authentication: Clerk

Team skill fit: Clerk is seamlessly integrated into Next.js and takes away from us much of the security-sensitive code concerning identities. **Licensing:** Proprietary, service is utilized under service license terms. **Community Support:** Clerk has Next.js SDKs and keeps its documentation up-to-date. **Performance:** Identity and sessions are managed by an identity service, which is dedicated to this purpose, while TradeIO continues with its enforcement of ownership and permissions on the server-side. **Cost Hosting:** As a hosted solution, we do not have to run our own identity server, and our anticipated use for the capstone is well within our budget. We did not want to reinvent the wheel of authentication, which is not TradeIO's core purpose.

Hosting: Vercel and Neon

Team skill fit: Our choice of technologies is based on Next.js, while Neon integrates seamlessly with Postgres and our TypeScript back end. **Licensing:** The platforms themselves are employed under the terms of their respective services, while the TradeIO source code remains within the possession of our team. **Community support:** They both have strong deployment guides and good examples. **Performance:** Vercel is suited for the interactive web application, and Neon provides us with relational storage in the cloud with indexed queries. **Cost and Hosting:** Both platforms provide free and inexpensive tiers to cover development during senior design, while we can scale up if necessary. We dismissed the option to host our own VM due to the overhead required by TLS, patching, process management, backups, and database management.

External AI and Notification Services

Team skill fit: The encapsulation of all external services using an adapter ensures that there will be no provider-specific code in C4 and C5. **Licensing:** Every provider will be used according to their API and service conditions. **Community support:** We will select providers having documentation as well as working API. **Performance:** No calls to AI and notification services are made within the correctness critical database transaction, hence, in case any of these fails, trade is guaranteed to persist along with the deterministic analytics. **Cost and Hosting:** Adapters will enable us to use any service we like according to our budget and rate limit.

7. D0-to-D1 Traceability

D0 Element	D1 Detail
C4 Risk & Behavior Analysis	Algorithms A and B, analysis output types, stored analysis records, and analysis API behavior.
C5 Data Storage	Relational entities, primary/foreign keys, relationships, indexes, and PostgreSQL choice.
I3 Trade & Journal Processing ↔ Data Storage	Trade and journal persistence structures and storage-failure behavior.
I4 Data Storage ↔ Risk & Behavior Analysis	Indexed trade/journal/settings retrieval and stored analysis results.
I5 Risk & Behavior Analysis → Application/API	Analysis response shape containing metrics, alerts/flags, and behavior summary.
I8 Risk & Behavior Analysis ↔ AI Insight Service	Deterministic behavior summary can be passed to the optional AI service; core analytics do not depend on AI.
I9 User Interface → Application/API	Risk settings, time zone, and currency are represented in the relational model and validated API inputs.
I10 Application/API → Notification Service	Notification delivery remains external and does not alter core trade or analysis records.